AI 3453 Deep Learning

Deep Learning

Convolutional Neural Networks

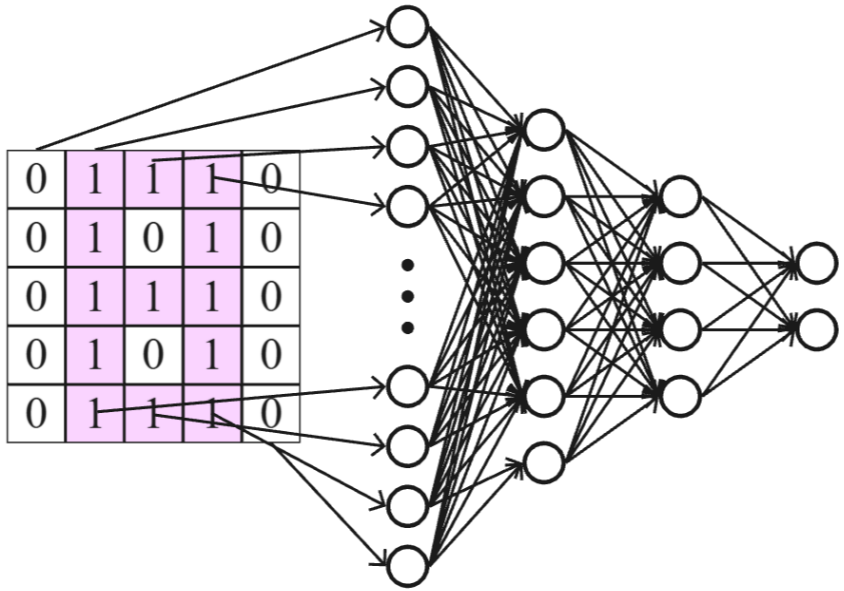
Temsiririrkkul Sila Ph.D.

Convolutional Network

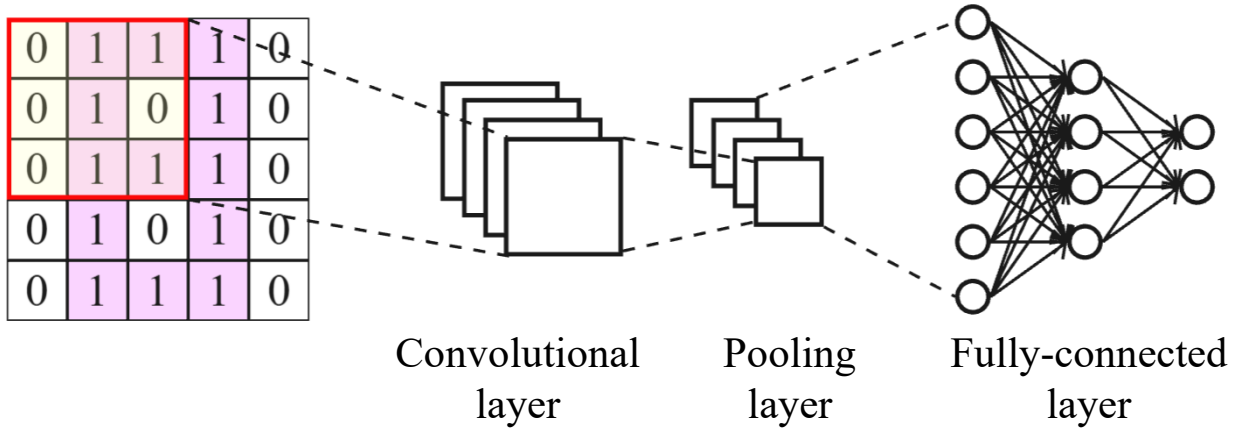
- Also known as CNN, CNNs.
- CNNs is a class of Neural Networks that specializes in processing data that has a grid-like topology, such as an image.
- There are three mains type of layers:
    1. Convolutional layer
    2. Pooling layer
    3. Fully-Connected layer

CNN Architecture

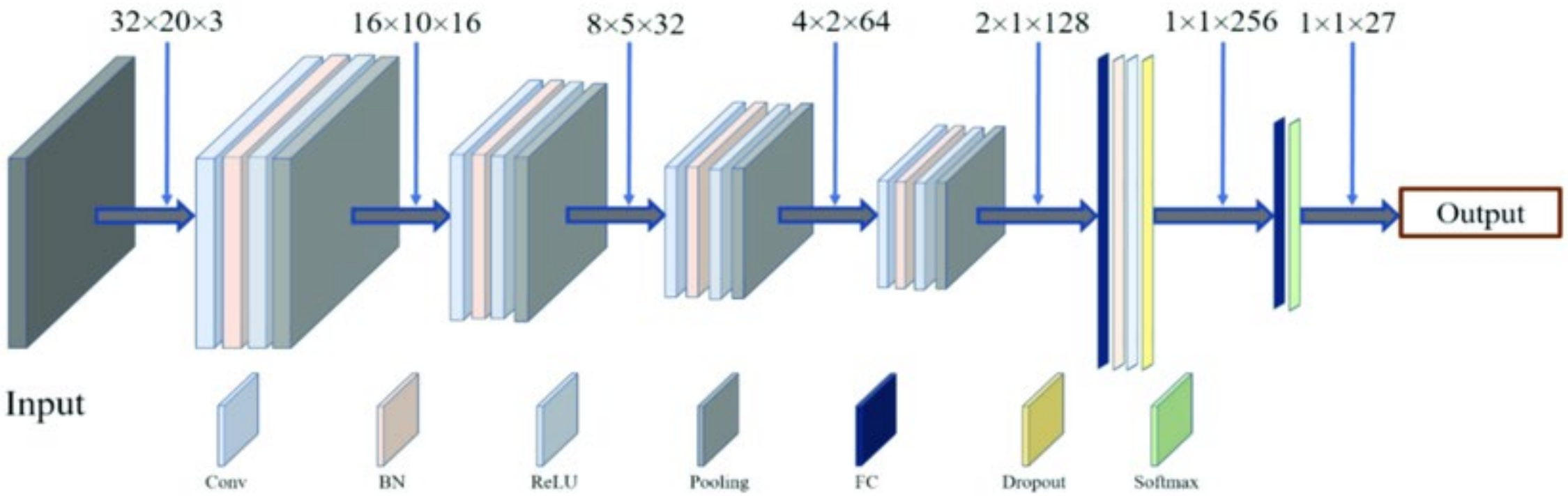
ANN



CNN



DCNN Architecture



Convolutional Layer

- performs a dot product between two matrices, where one matrix is the set of learnable parameters otherwise known as a kernel, and the other matrix is the restricted portion of the receptive field.
- There are three components:
  - Filter size
  - Stride
  - Padding

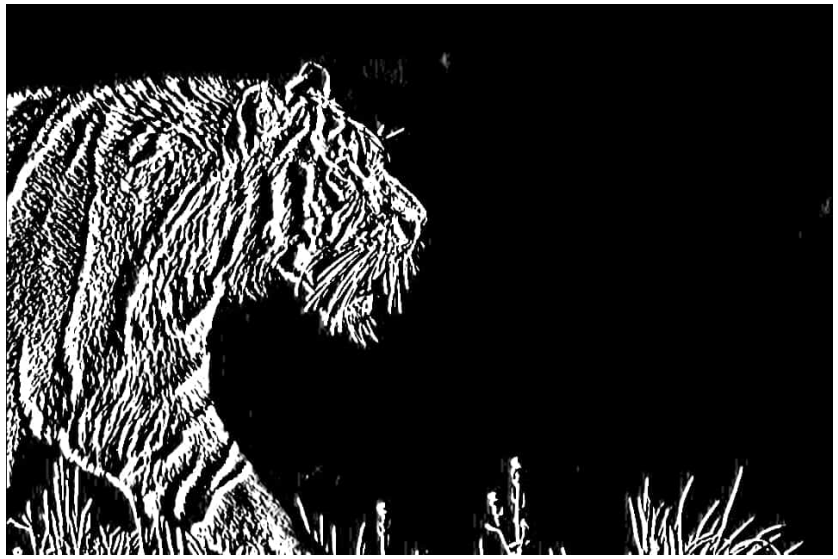
Filter

- Convolution filters are a powerful technique for extracting features from images and are widely used in various applications such as image enhancement, object detection, and image recognition.
- As described above we can broadly divide the main filters into three categories:
  - Edge detection filters (Sobel, Scharr, Canny)
  - Blur filters (Gaussian, Median, Bilateral)
  - Sharpen filters (Laplacian, Unsharp Mask)

Edge detection filters

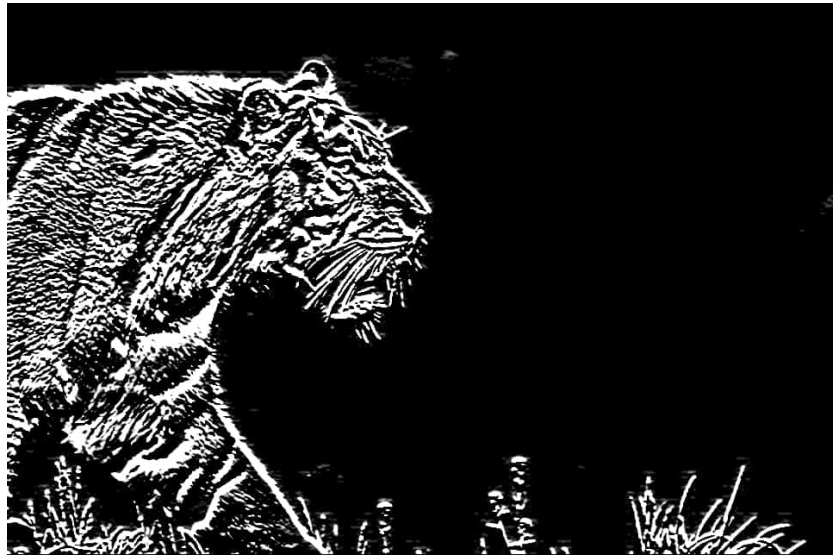
-1	0	1
-2	0	2
-1	0	1

Sobel-x



-1	-2	-1
0	0	0
1	2	1

Sobel-y



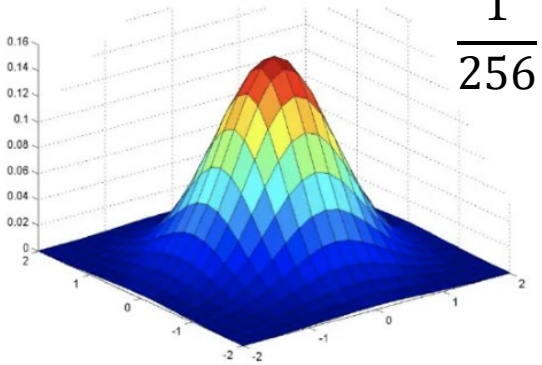
Blur filters

Box filter

$$\frac{1}{K^2}\begin{bmatrix} 1 & 1 & \dots & 1 \\ 1 & 1 & \dots & 1 \\ \dots & \dots & 1 & \dots \\ 1 & 1 & \dots & 1 \end{bmatrix}$$

Gaussian filter

$$\frac{1}{256}\begin{bmatrix} 1 & 4 & 6 & 4 & 1 \\ 4 & 16 & 24 & 16 & 4 \\ 6 & 24 & 36 & 24 & 6 \\ 4 & 16 & 24 & 16 & 4 \\ 1 & 4 & 6 & 4 & 1 \end{bmatrix}$$



Sharpen filters

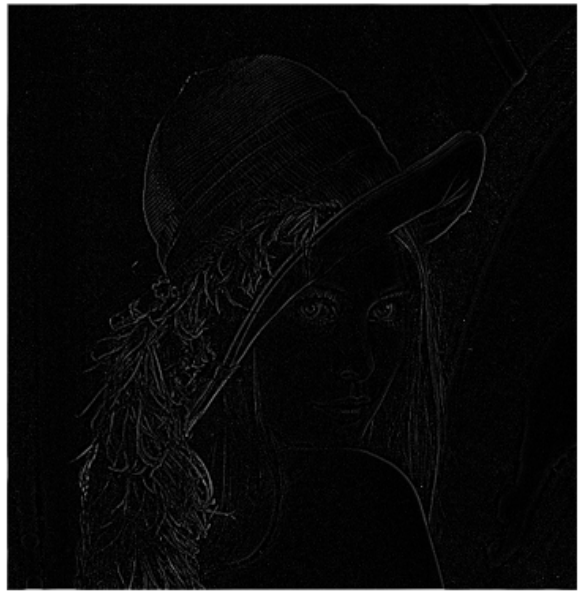
Laplacian Filter

0	-1	0
-1	4	-1
0	-1	0

Kernel 1

-1	-1	-1
-1	8	-1
-1	-1	-1

Kernel 2



Kernel 1

Kernel 2

Filter size

- Refers to width and height of the filter.
- We usually used size 3 x 3.
- Size 1 x 1 was only used for dimensionality reduction that aims to reduce the number of channels.
- Size 2 x 2 or 4 x 4 were not preferred because odd-sized filters symmetrically divide the previous layer pixels around the output pixel.

Stride

- Was the distance or the number of pixels, that the kernel moves over the input matrix.
- A larger stride yields a smaller output.

Convolution

$$g(i,j) = \sum_{u=-n}^{n} \sum_{v=-n}^{n} I(i-u, j-v) H(u,v)$$

0	1	0	1	0
0	1	0	1	0
0	0	0	0	0
1	0	0	0	1
0	1	1	1	0

I

$\oplus$

0	0	0
1	1	1
0	0	0

K

=

G = I$\oplus$K

Convolution

(0*0)+(1*0)+(0*0)+
(0*1)+(1*1)+(0*1)+
(0*0)+(0*0)+(0*0)
= 1

0	1	0	1	0
0	1	0	1	0
0	0	0	0	0
1	0	0	0	1
0	1	1	1	0

I

$\oplus$

0	0	0
1	1	1
0	0	0

K

=

	1			

$I \oplus K$

Convolution

(1*0)+(0*0)+(1*0)+
(1*1)+(0*1)+(1*1)+
(0*0)+(0*0)+(0*0)
= 2

0	1	0	1	0
0	1	0	1	0
0	0	0	0	0
1	0	0	0	1
0	1	1	1	0

I

$\oplus$

0	0	0
1	1	1
0	0	0

K

=

	1	2		

$I \oplus K$

Convolution

(0*0)+(1*0)+(0*0)+
(0*1)+(1*1)+(0*1)+
(0*0)+(0*0)+(0*0)
= 1

0	1	0	1	0
0	1	0	1	0
0	0	0	0	0
1	0	0	0	1
0	1	1	1	0

I

$\oplus$

0	0	0
1	1	1
0	0	0

K

=

	1	2	1	

$I \oplus K$

Convolution

(0*0)+(1*0)+(0*0)+
(0*1)+(0*1)+(0*1)+
(1*0)+(0*0)+(0*0)
= 0

0	1	0	1	0
0	1	0	1	0
0	0	0	0	0
1	0	0	0	1
0	1	1	1	0

I

$\oplus$

0	0	0
1	1	1
0	0	0

K

=

	1	2	1	
	0			

I$\oplus$K

Convolution

(1*0)+(0*0)+(1*0)+
(0*1)+(0*1)+(0*1)+
(0*0)+(0*0)+(0*0)
= 0

0	1	0	1	0
0	1	0	1	0
0	0	0	0	0
1	0	0	0	1
0	1	1	1	0

I

$\oplus$

0	0	0
1	1	1
0	0	0

K

=

	1	2	1	
	0	0		

$I \oplus K$

Convolution

(0*0)+(1*0)+(0*0)+
(0*1)+(0*1)+(0*1)+
(0*0)+(0*0)+(1*0)
= 0

0	1	0	1	0
0	1	0	1	0
0	0	0	0	0
1	0	0	0	1
0	1	1	1	0

I

$\oplus$

0	0	0
1	1	1
0	0	0

K

=

	1	2	1	
	0	0	0	

$I \oplus K$

Convolution

(0*0)+(0*0)+(0*0)+
(1*1)+(0*1)+(0*1)+
(0*0)+(1*0)+(1*0)
= 1

0	1	0	1	0
0	1	0	1	0
0	0	0	0	0
1	0	0	0	1
0	1	1	1	0

I

$\oplus$

0	0	0
1	1	1
0	0	0

K

=

	1	2	1	
	0	0	0	
	1			

I$\oplus$K

Convolution

(0*0)+(0*0)+(0*0)+
(0*1)+(0*1)+(0*1)+
(1*0)+(1*0)+(1*0)
= 0

0	1	0	1	0
0	1	0	1	0
0	0	0	0	0
1	0	0	0	1
0	1	1	1	0

I

$\oplus$

0	0	0
1	1	1
0	0	0

K

=

	1	2	1	
	0	0	0	
	1	0		

$I \oplus K$

Convolution

(0*0)+(0*0)+(0*0)+
(0*1)+(0*1)+(1*1)+
(1*0)+(1*0)+(0*0)
= 1

0	1	0	1	0
0	1	0	1	0
0	0	0	0	0
1	0	0	0	1
0	1	1	1	0

I

$\oplus$

0	0	0
1	1	1
0	0	0

K

=

	1	2	1	
	0	0	0	
	1	0	1	

$I \oplus K$

Convolution

What happened?

0	1	0	1	0
0	1	0	1	0
0	0	0	0	0
1	0	0	0	1
0	1	1	1	0

I

$\oplus$

0	0	0
1	1	1
0	0	0

K

$=$

1	2	1
0	0	0
1	0	1

$I \oplus K$

Padding

- Was usually used when the filter do not fit the input image.
- This set all elements that fall outside of the input matrix to zero.
- There are three type of paddings:
  1. Valid padding: also known as no padding.
  2. Same padding: ensures the output layer has the same size as the input layer.
  3. Full padding: increases the size of the output by adding zero to the border of the input.

Zero Padding

0	0	0	0	0	0	0
0	0	1	0	1	0	0
0	0	1	0	1	0	0
0	0	0	0	0	0	0
0	1	0	0	0	1	0
0	0	1	1	1	0	0
0	0	0	0	0	0	0

I

$\oplus$

0	0	0
1	1	1
0	0	0

K

=

$I \oplus K$

Zero Padding

0	0	0	0	0	0	0
0	0	1	0	1	0	0
0	0	1	0	1	0	0
0	0	0	0	0	0	0
0	1	0	0	0	1	0
0	0	1	1	1	0	0
0	0	0	0	0	0	0

I

$\oplus$

0	0	0
1	1	1
0	0	0

K

$=$

1	1	2	1	1
1	1	2	1	1
0	0	0	0	0
1	0	0	0	1
1	2	3	2	1

$I \oplus K$

Pooling layer

- Where Convolutional layers extract features from images, Pooling layers consolidate the features learned by CNNs.
- There are two types of poolings that are used:
  - Max pooling: selecting the maximum value from every pool.
  - Average pooling: getting the average of the pool.

Max Pooling

1	1	2	1	1
1	1	2	1	1
0	0	0	0	0
1	0	0	0	1
1	2	3	2	1

1			

1	1	2	1	1
1	1	2	1	1
0	0	0	0	0
1	0	0	0	1
1	2	3	2	1

1	2		

Max Pooling

1	1	2	1	1
1	1	2	1	1
0	0	0	0	0
1	0	0	0	1
1	2	3	2	1

1	2	2	

1	1	2	1	1
1	1	2	1	1
0	0	0	0	0
1	0	0	0	1
1	2	3	2	1

1	2	2	1

Max Pooling

1	1	2	1	1
1	1	2	1	1
0	0	0	0	0
1	0	0	0	1
1	2	3	2	1

1	2	2	1
1			

1	1	2	1	1
1	1	2	1	1
0	0	0	0	0
1	0	0	0	1
1	2	3	2	1

1	2	2	1
1	2		

Max Pooling

1	1	2	1	1
1	1	2	1	1
0	0	0	0	0
1	0	0	0	1
1	2	3	2	1

1	2	2	1
1	2	2	

1	1	2	1	1
1	1	2	1	1
0	0	0	0	0
1	0	0	0	1
1	2	3	2	1

1	2	2	1
1	2	2	1

Max Pooling

1	1	2	1	1
1	1	2	1	1
0	0	0	0	0
1	0	0	0	1
1	2	3	2	1

1	2	2	1
1	2	2	1
1			

1	1	2	1	1
1	1	2	1	1
0	0	0	0	0
1	0	0	0	1
1	2	3	2	1

1	2	2	1
1	2	2	1
1	0		

Max Pooling

1	1	2	1	1
1	1	2	1	1
0	0	0	0	0
1	0	0	0	1
1	2	3	2	1

1	2	2	1
1	2	2	1
1	0	0	

1	1	2	1	1
1	1	2	1	1
0	0	0	0	0
1	0	0	0	1
1	2	3	2	1

1	2	2	1
1	2	2	1
1	0	0	1

Max Pooling

1	1	2	1	1
1	1	2	1	1
0	0	0	0	0
1	0	0	0	1
1	2	3	2	1

1	2	2	1
1	2	2	1
1	0	0	1
2			

1	1	2	1	1
1	1	2	1	1
0	0	0	0	0
1	0	0	0	1
1	2	3	2	1

1	2	2	1
1	2	2	1
1	0	0	1
2	3		

Max Pooling

1	1	2	1	1
1	1	2	1	1
0	0	0	0	0
1	0	0	0	1
1	2	3	2	1

1	2	2	1
1	2	2	1
1	0	0	1
2	3	3	

1	1	2	1	1
1	1	2	1	1
0	0	0	0	0
1	0	0	0	1
1	2	3	2	1

1	2	2	1
1	2	2	1
1	0	0	1
2	3	3	2

Average Pooling

1	1	2	1	1
1	1	2	1	1
0	0	0	0	0
1	0	0	0	1
1	2	3	2	1

1			

1	1	2	1	1
1	1	2	1	1
0	0	0	0	0
1	0	0	0	1
1	2	3	2	1

1	1.5		

Average Pooling

1	1	2	1	1
1	1	2	1	1
0	0	0	0	0
1	0	0	0	1
1	2	3	2	1

1	1.5	1.5	

1	1	2	1	1
1	1	2	1	1
0	0	0	0	0
1	0	0	0	1
1	2	3	2	1

1	1.5	1.5	1

Average Pooling

1	1	2	1	1
1	1	2	1	1
0	0	0	0	0
1	0	0	0	1
1	2	3	2	1

1	1.5	1.5	1
0.5			

1	1	2	1	1
1	1	2	1	1
0	0	0	0	0
1	0	0	0	1
1	2	3	2	1

1	1.5	1.5	1
0.5	0.75		

Average Pooling

1	1	2	1	1
1	1	2	1	1
0	0	0	0	0
1	0	0	0	1
1	2	3	2	1

1	1.5	1.5	1
0.5	0.75	0.75	

1	1	2	1	1
1	1	2	1	1
0	0	0	0	0
1	0	0	0	1
1	2	3	2	1

1	1.5	1.5	1
0.5	0.75	0.75	0.5

Average Pooling

1	1	2	1	1
1	1	2	1	1
0	0	0	0	0
1	0	0	0	1
1	2	3	2	1

1	1.5	1.5	1
0.5	0.75	0.75	0.5
0.25			

1	1	2	1	1
1	1	2	1	1
0	0	0	0	0
1	0	0	0	1
1	2	3	2	1

1	1.5	1.5	1
0.5	0.75	0.75	0.5
0.25	0		

Average Pooling

1	1	2	1	1
1	1	2	1	1
0	0	0	0	0
1	0	0	0	1
1	2	3	2	1

1	1.5	1.5	1
0.5	0.75	0.75	0.5
0.25	0	0	

1	1	2	1	1
1	1	2	1	1
0	0	0	0	0
1	0	0	0	1
1	2	3	2	1

1	1.5	1.5	1
0.5	0.75	0.75	0.5
0.25	0	0	0.25

Average Pooling

1	1	2	1	1
1	1	2	1	1
0	0	0	0	0
1	0	0	0	1
1	2	3	2	1

1	1.5	1.5	1
0.5	0.75	0.75	0.5
0.25	0	0	0.25
1			

1	1	2	1	1
1	1	2	1	1
0	0	0	0	0
1	0	0	0	1
1	2	3	2	1

1	1.5	1.5	1
0.5	0.75	0.75	0.5
0.25	0	0	0.25
1	1.25		

Average Pooling

1	1	2	1	1
1	1	2	1	1
0	0	0	0	0
1	0	0	0	1
1	2	3	2	1

1	1.5	1.5	1
0.5	0.75	0.75	0.5
0.25	0	0	0.25
1	1.25	1.25	

1	1	2	1	1
1	1	2	1	1
0	0	0	0	0
1	0	0	0	1
1	2	3	2	1

1	1.5	1.5	1
0.5	0.75	0.75	0.5
0.25	0	0	0.25
1	1.25	1.25	1

Pooling

1	1	2	1	1
1	1	2	1	1
0	0	0	0	0
1	0	0	0	1
1	2	3	2	1

Max Pooling

1	2	2	1
1	2	2	1
1	0	0	1
2	3	3	2

Average Pooling

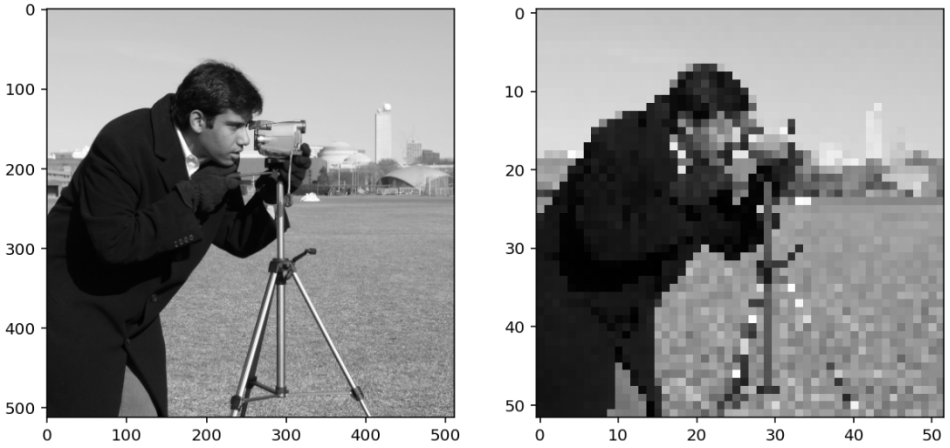
1	1.5	1.5	1
0.5	0.75	0.75	0.5
0.25	0	0	0.25
1	1.25	1.25	1

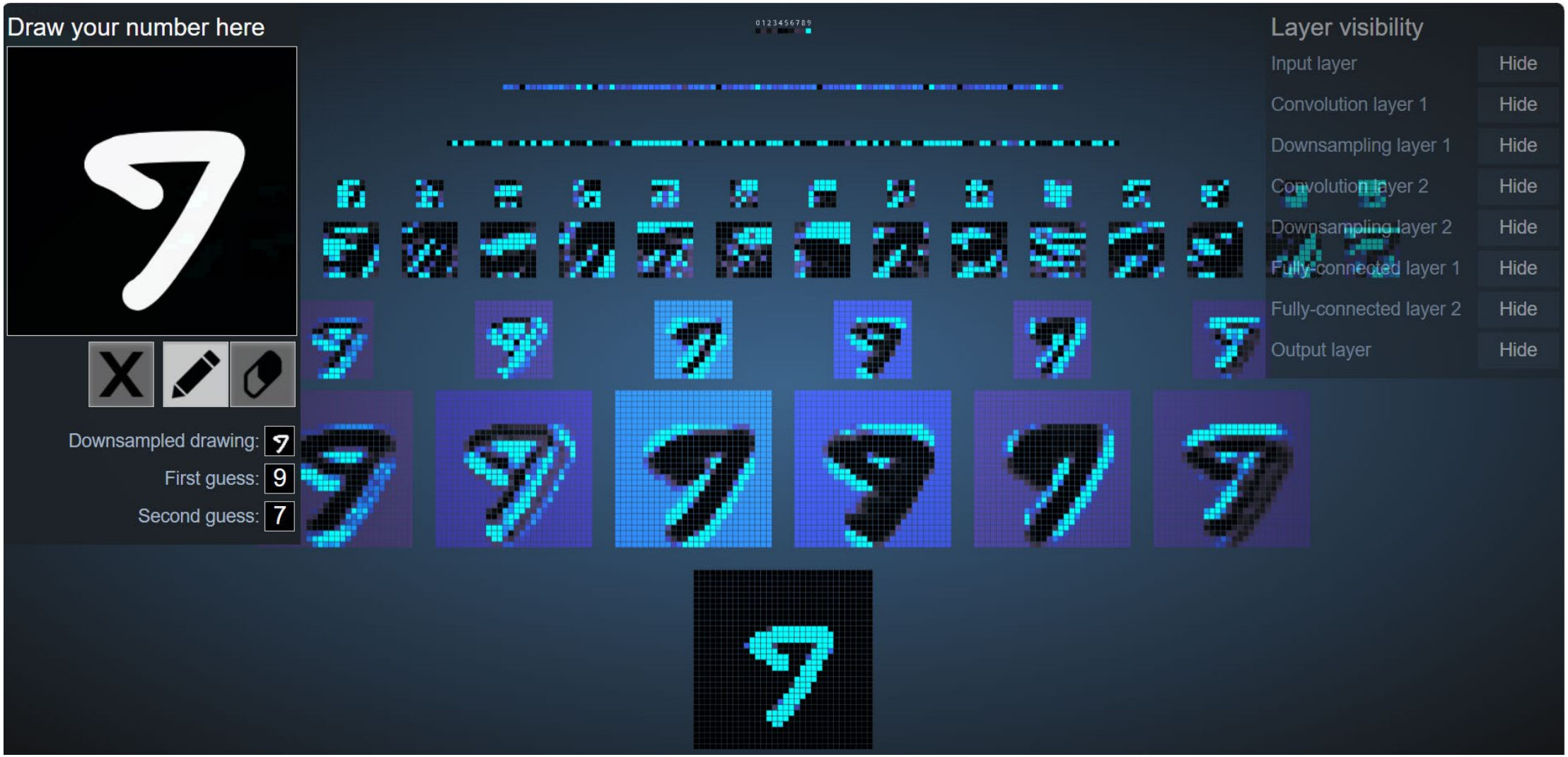
Fully-Connected Layer

- Fully connected input layer (flatten) – takes the output of the previous layers, "flattens" them and turns them into a single vector that can be an input for the next stage.

Down Sampling Process

- In DCNN we convolute Input data where represent in multiple dimension matrix.
- The feed forward process in convolute layer through pulling layer are downsizing the number of input size sometime call Down sampling process





https://adamharley.com/nn_vis/

0	0	0	0	1	1	1	1	0
0	0	1	1	1	0	0	1	0
0	1	1	0	0	0	1	0	0
0	0	0	0	0	0	1	0	0
0	0	0	0	0	1	0	0	0
0	0	0	0	1	0	0	0	0
0	0	0	1	0	0	0	0	0
0	0	1	0	0	0	0	0	0
0	1	0	0	0	0	0	0	0

Filter

0	1	0
1	1	1
0	1	0

-1	0	1
-2	0	2
-1	0	1

Zero padding = 2
Stride = 2
Pooling:

- max pooling 2x2
- stride 1